PHASE 2 — ELITE INTEGRATION ENGINEERING

W1D1 → W24D6 (24 Weeks)

CORE PROJECT:

“University Integration Platform (BC Integration Layer)”

You will build:

- External API consumption engine
- BC API exposure layer
- OAuth2 authentication system
- Webhook system
- File integration engine
- Full logging + retry framework

PHASE STRUCTURE RULE

Each 6-week block = one integration maturity level:

Block	Focus
W1–W6	REST API consumption engine
W7–W12	BC as API provider
W13–W18	OAuth2 + Webhooks
W19–W24	Enterprise integration framework

BLOCK 1 — W1D1 → W6D6

REST API CONSUMPTION ENGINE

W1D1 — HttpClient Fundamentals

Theory

- HttpClient lifecycle
- Request/Response model

Build

- Create Integration Codeunit:
    - Base HTTP service wrapper

W1D2 — GET Requests

Build

- Consume public API (students/JSON placeholder style)
- Return raw response

W1D3 — Response Parsing

Theory

- JSON structure understanding

Build

- Parse JsonObject → fields
- Extract structured data

W1D4 — Data Mapping Layer

Build

- Map external JSON → BC Student temp record

W1D5 — Error Handling Basics

Build

- Handle:
    - timeout
    - invalid JSON
    - failed response codes

W1D6 — BLOCK TEST 1 (GATE 1)

TEST

- Call external API
- Parse response
- Store in BC table

PASS CRITERIA

- Stable API consumption
- No crashes on invalid data

BLOCK 1 REVIEW WEEK COMPLETE

W2D1 — POST Requests

Build

- Send Student data to external API

W2D2 — Request Builder Pattern

Build

- Central request builder codeunit

W2D3 — Headers & Authentication basics

Build

- Add API keys in headers

W2D4 — Retry Logic v1

Build

- Retry failed calls (simple loop)

W2D5 — Logging Integration Start

Build

- Log API requests/responses

W2D6 — TEST GATE 2

TEST

- POST data externally
- Validate retry works
- Validate logs exist

BLOCK 2 — BC AS API PROVIDER (W7D1 → W12D6)

W7D1 — API Pages Deep Dive

Build

- Enhance Student API page

W7D2 — CRUD API Expansion

Build

- Full create/update/delete via API

W7D3 — Data Validation Layer

Build

- Validate API input before insert

W7D4 — Versioning Concept

Build

- v1 API structure

W7D5 — External System Simulation

Build

- Simulate external system calls

W7D6 — TEST GATE 3

TEST

- External system creates BC student
- Updates student
- Reads student

BLOCK 3 — AUTHENTICATION & SECURITY (W13D1 → W18D6)

W13D1 — OAuth2 Theory

Understand flows

W13D2 — Token Handling

Build

- Token storage table

W13D3 — Token Refresh Logic

Build

- Auto-refresh expired token

W13D4 — Secure API Calls

Build

- Attach bearer tokens

W13D5 — Secret Management

Build

- Setup table for secrets

W13D6 — TEST GATE 4

TEST

- OAuth authenticated API call works

BLOCK 4 — WEBHOOKS & EVENT DRIVEN SYSTEMS

W19D1 — Webhook Fundamentals

Theory

- Push vs pull systems

W19D2 — Webhook Receiver API

Build

- Incoming endpoint in BC

W19D3 — Event Triggering

Build

- Webhook triggers student update

W19D4 — Validation Layer

Build

- Validate incoming payloads

W19D5 — Async Processing

Build

- Queue webhook processing

W19D6 — TEST GATE 5

TEST

- External system pushes update
- BC processes it automatically

BLOCK 5 — FILE INTEGRATION ENGINE

W20D1 — File Upload Handling

Build

- InStream file processing

W20D2 — CSV Parsing Engine

Build

- Parse bulk student uploads

W20D3 — Error Row Handling

Build

- Skip invalid rows, log errors

W20D4 — Export Engine

Build

- Export BC data to file

W20D5 — Scheduled Processing

Build

- Job queue for file imports

W20D6 — TEST GATE 6

TEST

- Upload bulk student file
- System processes correctly

BLOCK 6 — ENTERPRISE INTEGRATION FRAMEWORK

W21D1 — Architecture Design

Build

- Unified integration framework structure

W21D2 — Retry System v2

Build

- Exponential retry logic

W21D3 — Dead Letter Handling

Build

- Failed integration storage

W21D4 — Monitoring Layer

Build

- Integration dashboard data

W21D5 — Framework Refactor

Build

- Convert all integrations to framework

W21D6 — FINAL PHASE 2 TEST (GATE 7)

FINAL TEST REQUIREMENTS

You MUST demonstrate:

- External API consumption
- BC API exposure
- OAuth2 authentication
- Webhook processing
- File integration system
- Retry + logging framework

- Clean architecture

PHASE 2 EXIT CRITERIA

You only move to Phase 3 if:

✓ You can design integrations without help

✓ You have reusable integration framework

✓ You handle failures properly

✓ You understand system-level integration flow

✓ Your code is production-safe

IMPORTANT REALITY CHECK

If Phase 1 builds your:

“engineering foundation”

Then Phase 2 builds your:

“market value”

This is the phase your PM is indirectly targeting when they mention:

- APIs
- integrations
- middleware
- backend logic
- external systems